

Amazon Internship Interview Experience

-Kabileshwaran SD (2021115046)

About the company

Amazon is a global tech company that started as an online bookstore and grew into one of the largest e-commerce platforms. It lead in cloud computing with AWS, innovate with devices like Echo, and focus on customer obsession, operational excellence, and continuous innovation.

Role details Offered role:

SDE Intern (Software Development Engineer)
Tenure: 6 months

Preliminary Round

The preliminary round was conducted online using Hackerank's coding platform. In this round, there were 2 coding questions that needed to be solved within 1.5 hours

Question 1: Finding a Common Regex Pattern

Problem Description: You are tasked with developing a service that finds a common regex pattern from a given set of patterns. The regex patterns consist of lowercase English letters and wildcard characters (?) which can match any single English letter. Two patterns are said to intersect if you can replace the wildcard characters in both patterns to form the same string from both patterns.

The goal is to find the minimum number of wildcard characters (?) required in a pattern that intersects with all the given patterns.

Reference: https://www.glassdoor.ca/Interview/Amazon-is-developing-a-string-matching-library-and-you-are-to-develop-a-service-that-finds-a-common-regex-pattern-from-the-QTN_7518037.htm

Initially I tried to solve this in brute force only few test cases has been passed. Then I solved this using DP where all test cases has been passed.

Question 2: Minimum Network Lag

Problem Description: The second question was comparatively straightforward. It required finding the minimum network lag between given source and destination servers. Specifically, you are provided with two arrays: one representing data centers and the other representing destination servers. The task is to compute the minimum network lag.

Input: datacentres = [1, 2, 2], servers = [2, 4, 5]

Output: 6

Approach:

1. **Sorting:**
 - To minimize the total network lag, I sorted both the datacentre and destination servers arrays.
2. **Compute Lag:**
 - After sorting, I calculated the lag by summing the absolute differences between corresponding elements from the two sorted arrays.

In this example, after sorting:

- datacentres = [1, 2, 2]
- servers = [2, 4, 5]

The minimum network lag is calculated as:

- $|1 - 2| + |2 - 4| + |2 - 5| = 1 + 2 + 3 = 6$

I managed to solve both coding problems in the preliminary round within 25-30 minutes. After the coding test, there was a mandatory work-level assessment, consisting of around 60 questions focused on Amazon's Leadership Principles. Many candidates mentioned that they completed both coding problems but chose to skip this work-level assessment, which resulted in them not being shortlisted. Therefore, it is crucial to complete the work-level assessment to proceed in the selection process.

For more information on Amazon's Leadership Principles, you can refer to this [link](#).

Technical Interview:

Around 400 students attended the online test, and out of those, 87 students were selected for the next round. We received a 1-hour time slot for an online interview the day before, which took place on Amazon Chime. At the start of the interview, the interviewer sent a code editor link via email where the coding question was pasted. I was required to write my solution directly in that code editor.

The interviewer, an SDE-II from Amazon's Bangalore office with 4.5 years of experience, briefly introduced himself and asked for my introduction. After I introduced myself, the interviewer moved on to the coding question.

Coding Question 1: Task Scheduling

The first question was about task scheduling. Given a number of tasks N and task dependencies, where $[T1, T2]$ means Task 2 needs to be completed before Task 1, and $[T1, T3]$ means Task 3 needs to be completed before Task 1, you were to find a valid order of

tasks. This problem is essentially a topological sort. This question is similar to the "Course Schedule" problem on LeetCode.

I explained Kahn's algorithm for topological sorting and demonstrated a dry run with a provided test case. I also noted that the algorithm would fail if there were cyclic dependencies among the tasks. After the dry run, the interviewer asked me to code the solution. Once I completed the coding, we did another dry run together.

After coding, the interviewer asked about the space and time complexity of the solution. I explained that the time complexity is $O(V+E)$ where V is the number of vertices (tasks) and E is the number of edges (dependencies). I elaborated on how this complexity is derived from the algorithm's operations. The interviewer also inquired about optimizing the solution, and I mentioned the possibility of executing tasks in parallel using threads.

Coding Question 2: Trapping Rain Water

The next question involved finding the amount of water that can be trapped between buildings of different heights after a rain. This question is similar to the "Trapping Rain Water" problem on LeetCode.

Initially, I provided a solution with space complexity of $O(2n)$ and time complexity of $O(n)$ and then I wrote the code. The interviewer then asked me to optimize the space complexity to $O(1)$. I explained the two-pointer approach to achieve this and the interviewer was satisfied with the explanation.

Conclusion

At the end of the interview, the interviewer asked if I had any questions. Since I had interned at Amazon for two months, I was familiar with the work culture and related aspects, so I said I had no questions. The interviewer wished me all the best and ended the meeting 5 minutes before the scheduled end time.

Tips:

- Always start by explaining the brute-force solution, even if you already know the optimal approach as they will assess your thought process too.
- Stay engaged during the interview; avoid being silent.
- Be clear while explaining your thought process/approach

Preparation Docs:

- ❖ To cover all patterns, I followed Striver's A2Z DSA Sheet. For example, when learning graphs, start with BFS, DFS, and distance algorithms, then practice related problems on LeetCode and GeeksforGeeks.

- ❖ Develop a strong understanding of time and space complexity.
- ❖ Learn various data structures and their standard problems.
- ❖ If you can't immediately solve a problem, discuss your approach and thought process.
- ❖ Focus on solving a diverse range of problems rather than just a high volume of similar ones. This strategy prepares you for unexpected questions and enhances your understanding of DSA concepts.

All the very best! Feel free to reach out to me if you have any doubts.

- **LinkedIn Profile:** [Kabileshtwaran SD](#)
- **Contact Number:** +91 6380178240
- **Email:** kabilsrinivasan2004@gmail.com